

Annex XVIII — WR Connect™

Governed Publisher Semantics and Personal Compute Orchestration

Version 1.3 — 14 August 2026 · Part of the Optirando™ Canon · Status: Defensive Publication / Prior-Art Disclosure

Annex number provisional — assign per Canon sequence.

§XVIII.0 Purpose, Status, and Claims Discipline

§XVIII.0.1 Purpose

This annex discloses how a verified publisher can provide the meaning of its own application — its regions, entities, navigation, workflows, capabilities, and interface anchors — as governed, declarative, cryptographically committed material, and how a local user orchestrator consumes that material to assist, navigate, automate, and dispatch computation without the publisher gaining insight into the user's private context.

A central design goal runs through the entire annex: enabling low-power local AI to operate accurately. The publisher side applies strong reasoning once, during preparation, to optimize the semantic environment precisely for consumption by lightweight local models — so that accuracy on the user's device comes from the prepared environment, not from local model size.

§XVIII.0.2 Claims Discipline

This annex is a defensive prior-art disclosure, not a novelty claim. To the author's best knowledge, the individual technologies combined here exist in prior work — including publisher-declared machine semantics, capability-declaration protocols for AI clients, centrally maintained UI semantics in automation tooling, guidance overlays, hybrid on-device/cloud model execution, and computation offloading to nearby or stronger compute. No novelty is claimed for any individual element, and no element-by-element differentiation against specific prior systems is asserted anywhere in this document; this single acknowledgment covers the entire annex.

The disclosed contribution is not any individual element. It is the combination of these elements under a single cryptographic proof discipline — WR Handshake™, BEAP™, PoAC™, PoAE™ — with four authorities that never collapse into one another: the publisher is authoritative for declared meaning, the user orchestrator for private meaning, the dispatcher for execution location, and the local trusted UI for everything the user sees.

§XVIII.0.3 Amendment Discipline

Amendments follow the Canon-wide rule: versioned updates, never silent replacement; changes to invariants are explicit and reasoned in the Version History (§XVIII.13).

§XVIII.1 Problems Addressed and Value Statement

This section states the concrete problems the architecture solves, before any mechanism is defined. The scenarios are stated briefly in §XVIII.8; the technical realization is specified in §§XVIII.3–XVIII.7.

§XVIII.1.1 Support by local AI instead of hosted support chat

Today, application support is either human (expensive, slow) or a publisher-hosted chatbot (every question, and often the user's account context, flows to the publisher or its AI vendor). With a Publisher Semantic Layer, the publisher describes once — authoritatively — what its application means and how its workflows are structured. The user's local AI answers support questions and guides the user through the application using verified publisher semantics, on the user's device. The publisher reduces support cost without operating a support AI; the user receives assistance without disclosing questions, history, or account context to anyone.

§XVIII.1.2 Verified navigation assistance

"Show me where I can download my invoice." "How do I get to returns?" Today an assistant must guess this from pixels or DOM structure, and guesses wrong. With declared semantics, the WR Pointer™ resolves a semantic target from the publisher's own declaration and the WR Overlay™ marks the path — deterministically, from the authoritative source, with a fail-closed rule when the live interface and the declared semantics diverge (§XVIII.6.3).

§XVIII.1.3 Contact channel by BEAP instead of e-mail and web forms

Contact forms and contact e-mail addresses are today's dominant inbound fraud surface: spam, phishing, payload attachments, spoofed senders. A publisher that declares a contact capability in its semantic layer receives inbound contact as BEAP capsules over the handshake relationship instead: text-pure by construction (no executable payloads can exist in BEAP text), sender-bound to a handshake identity class (ID0–ID3 per Annex XIV), and screened by Fraud Watchdog™ on ingest. Phishing risk is mitigated structurally, by consistent out-of-band routing — contact travels over the sealed handshake relationship instead of clickable mail and open web forms — combined with Fraud Watchdog™ analysis of every inbound item. The user gains a verified channel to a cryptographically identified publisher; the publisher gains structured, classified, fraud-screened inbound and can retire its most abused legacy channel.

§XVIII.1.4 Conversion through declared registration and low-friction workflows

Registration friction destroys conversion. The user addressed here already holds a WR account but has no account with the publisher. Conventionally, registering means typing personal information into a form and then clicking a link in a confirmation e-mail. Both steps can be performed by the

orchestrator on the user's behalf: the publisher declares a registration capability with typed, declared operator-input slots; the orchestrator resolves those slots from the user's private context under identifier-only slot discipline and, after a single consent, submits them and completes the confirmation step — the confirmation e-mail arrives through the governed depackaging path, and its completion is a declared, consented action, never a hidden link click. When the publisher is itself in the WR system, the flow collapses further: the cryptographic account binding of the Public Handshake™ (Annex XIV) and the declared semantic graph make registration fully deterministic — no form, no confirmation e-mail, one consented action end to end. The conversion benefit accrues to the publisher; the data-minimization benefit accrues to the user — the publisher receives exactly the declared slots, nothing else.

§XVIII.1.5 Maintenance economics

When a publisher changes its interface, today every automation, every client-side agent, and every user re-derives meaning independently — or breaks. Here, the semantic representation is corrected once, at the source, published as a new catalog epoch, and propagated to all consumers. Semantic maintenance cost becomes $O(1)$ per interface change instead of $O(\text{consumers})$.

§XVIII.1.6 Accessibility (informative)

A Publisher Semantic Layer is, functionally, a signed, publisher-authoritative accessibility tree plus workflow state. Voice-driven navigation over verified rather than guessed semantics is a direct accessibility capability and is noted here as an intended application domain.

§XVIII.2 Definitions

WR Connect™. The collective designation for the architecture disclosed in this annex: the governed publisher connection through which declared application semantics, Context Frames, and the contact and registration capabilities of §XVIII.1 flow between a verified publisher and the local user orchestrator. The publisher-side WR Connect™ script (wr-connect) is the reference embodiment of the attested publisher channel on ordinary web hosting; it is an embodiment, not the definition — any implementation satisfying the properties of this annex is WR Connect™.

Publisher Semantic Layer (PSL). A declarative, machine-readable representation of the meaning of a publisher's application: semantic regions, entities and relations, navigation structure and transitions, workflow states, capabilities and their preconditions, Semantic Anchors and positional references, the Knowledge Corpus and Reasoning Units of §XVIII.3.7, and permitted WR Overlay™ declarations. A PSL is publisher material in the full Canon sense (§XVIII.3.1). The PSL is a generalization of the Scope Graph of Annex XIV; it introduces no parallel distribution, versioning, or trust mechanism.

Semantic Anchor. A declared reference to a UI element or region within the publisher's own application, used by the WR Pointer™ and WR Overlay™. Terminology note: a Semantic Anchor is distinct from the WR Anchor of Annex VIII, which designates an export integration point on a target page. The two terms are intentionally different and must not be conflated.

Semantic Chunk. An atomic, self-contained, declarative statement derived from the publisher's own application content, produced during preparation for retrieval-based consumption on the

visitor's device. Semantic Chunks form the Knowledge Corpus (Layer K, §XVIII.3.7). A chunk is publisher material in the full Canon sense and carries a provenance label like every other assertion.

Reasoning Unit. A prepared, provenance-labeled artifact capturing publisher-side reasoning about one visitor intent: the intent, retrieval paraphrases, preconditions, steps bound to declared structure, a grounded resolution, and references into the Knowledge Corpus. Reasoning Units supersede the Q&A scenario corpus of v1.0/v1.1 (§XVIII.3.7, Layer R).

Grounding Pack. The bounded, locally assembled set of retrieved material — Semantic Chunks, a matched Reasoning Unit, the relevant structural slice, and the active Context Frame — handed to the local model for answer formulation (§XVIII.4.5). A Grounding Pack is runtime material; it never leaves the user's trust domain.

Hybrid Retrieval. The local lookup discipline over the LSEM's indexed material combining a lexical channel and a semantic (embedding) channel (§XVIII.4.5). Both channels operate exclusively on-device.

Local Semantic Environment Model (LSEM). The persistent, local, provenance-classed graph in which the user orchestrator merges validated publisher semantics with locally derived and user-provided meaning (§XVIII.4). The LSEM never leaves the user's trust domain.

Provenance classes. PUBLISHER_DECLARED, PUBLISHER_AI_INFERRED, LOCAL_INFERRED, USER_PROVIDED, PLATFORM_ATTESTED (§XVIII.4.2).

Context Frame. A short-lived, signed, displayable runtime object describing the currently active semantic context of a session (e.g., "checkout of verified publisher P, step 2 of 4"). Context Frames travel over the handshake channel and are never catalog artifacts (§XVIII.3.5).

WR Overlay™. The branded designation for the orchestrator-rendered assistance and automation surface presented over or alongside an application. Technically, this surface is what the Canon describes as the Augmented Overlay: "Augmented Overlay" remains the descriptive technical term for the construct, and WR Overlay™ is the brand name for it — chosen because it also conveys the governed automation the surface carries, not augmentation alone. Both terms refer to the same construct; wherever Canon documents use the technical term "Augmented Overlay", the branded designation WR Overlay™ applies equally. All properties, invariants, and governance defined for the Augmented Overlay (in particular Annex XI) apply to the WR Overlay™ unchanged.

WR Link™. A declared, consented automation trigger embedded in or referenced from publisher content: activating a WR Link™ executes a pre-declared AI automation capability on the page, based on WCR recordings, under full capability, consent, and PoAE™ governance. WR Link™ declarations are part of the PSL (§XVIII.3.7).

Personal Compute Dispatcher. The orchestrator component that places tasks among devices within the user's own or explicitly authorized compute environment (§XVIII.7). The governing principle is execution-location-aware orchestration: the interaction endpoint does not have to be the execution endpoint.

Interaction Endpoint / Execution Endpoint. The device the user interacts with, and the device on which a given task actually executes. The two may differ; the difference is governed, visible to the user, and invisible to publishers.

§XVIII.3 The Publisher Semantic Layer

§XVIII.3.1 Semantics as governed publisher material

A PSL is publisher material in the full sense of this Canon: signed under the publisher's key hierarchy, committed under the Catalog Head with anti-rollback (Annex XIV §XIV.5.5), countersigned at ingest into the Workflow Ready Cloud (WRC)[™], publicly auditable, and versioned by catalog epoch. Publication is attested handover per Annex XVII §XVII.3.3 — atomic per epoch, rejection never modification. No separate trust, distribution, or versioning mechanism exists for semantics. A semantic assertion carries the same evidentiary burden as an automation artifact.

§XVIII.3.2 Strictly declarative transport

The logical representation of a PSL is a graph. Its transport form is strictly declarative structured text within BEAP/pBEAP capsules, subject to BEAP text purity. A PSL **MUST NOT** carry executable content of any kind — no scripts, no WASM, no code in any encoding. The receiving orchestrator (1) receives, (2) validates, (3) admits under PoAC[™], (4) reconstructs the graph internally, and (5) merges it into the LSEM under §XVIII.4. Data and declarative semantics cross the boundary; code never does.

§XVIII.3.3 Asymmetric Preparation, free choice of authoring method, and review economics

Computationally expensive semantic analysis of an application — by strong publisher-side or cloud models — is performed once, on the publisher side, before interaction. Strong intelligence prepares the environment; lightweight local intelligence operates within the prepared environment. The output of that preparation crosses the trust boundary only in the form defined in §XVIII.3.2.

The purpose of this asymmetry is explicit, and v1.2 sharpens *where* the transferred value must land: a resource-constrained local model can operate accurately only if the prepared output is **consumable without heavy reasoning** on the device. Preparation output is therefore defined by its consumption mechanism (§XVIII.3.7): declared structure is consumed by deterministic lookup; declared knowledge is consumed by local retrieval over an indexed corpus; declared reasoning is consumed by retrieval plus instantiation. Prepared material that no local mechanism can retrieve or look up transfers no value regardless of its quality; producing such material is a preparation defect, not a local-model limitation. Open-ended interpretation, where small models fail, is turned into selection and lookup over validated declarations, where small models are reliable.

How a publisher produces and maintains its semantic layer is the publisher's free choice: manually authored, generated by publisher-side local AI, generated by cloud AI, or any mix of these — including manual authoring of critical declarations with AI-generated coverage of the rest. The governance is method-agnostic: signing, catalog commitment, ingest, countersignature, and audit apply identically regardless of how the artifacts were produced. The only normative consequence of the authoring method is provenance labeling: AI-generated semantics that the publisher has not reviewed **MUST** be labeled PUBLISHER_AI_INFERRED; semantics the publisher has authored,

or has explicitly reviewed and adopted as its own declaration, carry PUBLISHER_DECLARED. Presenting unreviewed AI inference as declaration violates provenance discipline.

Review economics (normative). Provenance labels are per-assertion; review *acts* are not. A single explicit operator act MAY adopt any set of AI-generated artifacts the operator has had the opportunity to inspect — including the complete output of a preparation run. Editing an artifact constitutes adoption of that artifact. Per-item adoption ceremony is not required by provenance discipline and MUST NOT be imposed by tooling as the only path; publishing unreviewed artifacts under their honest PUBLISHER_AI_INFERRED label remains legitimate. Accountability derives from the signature over the whole artifact set and the honesty of its labels, not from the granularity of the clicking.

Informative: the design goal of accuracy-by-preparation is normative in this annex; the magnitude of the resulting capability uplift of small local models is an empirical, measurable, and falsifiable thesis, not an architectural guarantee. A minimal falsification protocol is a paired benchmark: the same small local model answering a fixed set of realistic visitor questions against the live application, once without and once with the ingested and indexed layer, measured on retrieval hit rate, groundedness of answers, and navigation-target success.

§XVIII.3.4 Central maintenance, update, and invalidation

The publisher-side WR Connect™ script (an embodiment of the attested publisher channel, §XVIII.2) MAY observe the publisher's own application for structural change, regenerate affected semantic artifacts, and publish them as a new catalog epoch. Scalability, update behavior, versioning, cacheability, and invalidation are answered entirely by existing Canon mechanisms and are not redefined here: Catalog Head epochs with anti-rollback (versioning, invalidation), declared load classes entry-class/selection-class with the declared cache vocabulary (cacheability), and delta synchronization over the handshake channel (update propagation).

§XVIII.3.5 Static semantics versus live state

Static meaning — structure, navigation, capabilities, anchors — is catalog material under §XVIII.3.1. Live state — the user's current position in a workflow — is runtime material: short-lived, signed Context Frames over the handshake channel, mirroring the Canon's static/live boundary (Annex XIV). Context Frames confer no authority; they inform. Their availability follows the three-party availability model: without a live publisher orchestrator, static semantics remain fully usable from cache; Context Frames are simply absent.

§XVIII.3.6 Channel honesty

The channel carrying semantic updates and Context Frames is the handshake-bound direct channel. Where infrastructure permits, it is peer-to-peer; on constrained publisher hosting it is cryptographically the same sealed channel over a polled dumb carrier. This annex claims channel binding and sealing, not a universal P2P transport.

§XVIII.3.7 Preparation output inventory: three consumption-defined layers

The preparation process (§XVIII.3.3) produces a defined set of artifact classes, organized into three layers. The layer of an artifact class is determined by its consumption mechanism on the visitor's device — this binding is normative, because it is what makes the transferred value real for a small local model. Every item is a declarative, provenance-labeled, catalog-committed artifact class; the inventory is extensible per §XVIII.11, never at runtime.

Layer S — Structural Declarations. Consumed by deterministic lookup; no model in the loop.

1. **Navigation Graph** — nodes are semantic regions, pages, and workflow steps, each carrying a region classification (closed kind vocabulary per the semantic context registry, §XVIII.10) and a one-line purpose; edges are declared transitions.
2. **Entity and relation declarations** — objects of the application, their relations, and the preconditions of actions upon them.
3. **Capability declarations** — every offered capability, including the registration capability (§XVIII.1.4) and the BEAP contact form capability (§XVIII.1.3) with its typed slots and routing.
4. **Semantic Anchors and position markers** — the declared placement material for WR Overlay™ elements and navigation guidance (§XVIII.5.4). Anchors **MUST** be grounded: an anchor may only be declared for an element whose identifying feature (e.g. its exact visible text) is verifiably present in the captured application surface. AI-proposed anchors that fail this check are dropped fail-closed during preparation.
5. **Semantic navigation aids** — precomputed, declared routes to semantic targets ("path to returns", "path to invoice download"), consumed by the WR Pointer™. Every path element **MUST** reference a captured node; routes containing unverifiable elements are dropped fail-closed.
6. **WR Link™ declarations** — declared automations, bound to hash-identified WCR recordings, executable on the page via WR Link™ under capability, consent, and PoAE™ governance; loaded by reference at selection per the Canon's selection-scoped loading.
7. **Load, cache, and epoch declarations** — the existing Canon vocabulary (entry-class/selection-class, cache hints, catalog epoch) applied to all layers.

Layer K — Knowledge Corpus. Consumed by local hybrid retrieval (§XVIII.4.5). This layer carries the *content* of the application in retrieval-ready form; it is what grounds visitor questions about the application itself. v1.2 adds this layer explicitly: without it, the visitor-side model has no material to answer content questions from and must reason from zero — the exact failure the architecture exists to prevent.

8. **Semantic Chunks.** Each chunk is one atomic, self-contained declarative statement with the following normative fields:
 - **id** — content-addressed (hash over the canonical chunk fields); stable across epochs while the chunk is unchanged, so re-indexing on the device is incremental;
 - **text** — the statement itself. It **MUST** be understandable without the surrounding page (self-containment): pronouns resolved, implicit context made explicit;

- **kind** — closed vocabulary: `fact` | `policy` | `procedure` | `definition` | `attribute`;
- **aliases** — bounded list of alternative phrasings and synonyms for the chunk's subject; aliases exist for retrieval and are indexed alongside the text;
- **node_refs** — the graph nodes this chunk belongs to;
- **source** — the page path and the content hash of the page state the chunk was derived from;
- **provenance** label.

Chunks **MUST** be grounded exclusively in the publisher's captured content (never invented), **MUST NOT** contain executable content (§XVIII.3.2), and **SHOULD** each support at least one plausible visitor question — filler chunks are a preparation defect.

9. **FAQ set** — the publisher's curated FAQ as declared content, distinct from generated material by provenance label; indexed like chunks.

Layer R — Reasoning Units. Consumed by intent retrieval plus local instantiation. This layer supersedes the "Q&A scenario corpus" of v1.0/v1.1. Its purpose is to transfer *precomputed reasoning*, not question-answer surface forms: the strong preparation model does the hard thinking once; the local model finds the matching unit and instantiates it with private context.

10. Reasoning Units. Normative fields:

- **id** — content-addressed, stable while unchanged;
- **intent** — the visitor goal in canonical phrasing;
- **paraphrases** — bounded list (3–10) of realistic alternative phrasings of the same intent; this is the retrieval surface and is indexed;
- **preconditions** — declared conditions under which the unit applies (account state, prior steps, capability availability);
- **steps** — the ordered resolution path; each step **MAY** bind to a Semantic Anchor, a declared route, or a graph node by reference — steps that guide **MUST** bind to Layer S rather than describe locations in prose;
- **resolution** — the grounded outcome of the reasoning: what is true, what the visitor should conclude or do, stated so a small model can render it with private context; every factual claim in it **MUST** be supported by referenced chunks or declared structure;
- **pitfalls** — known edge cases and failure modes, each with its condition;
- **refs** — chunk ids, capability ids, and route ids this unit relies on;
- **provenance** label.

A Reasoning Unit whose resolution cannot be traced to its refs is invalid and **MUST** be rejected at preparation time.

Consumption binding (normative summary).

Layer	Artifact classes	Consumption on device	Model in loop
S	graph, entities, capabilities, anchors, routes, WR Links, load/epoch	deterministic lookup	no
K	Semantic Chunks, FAQ	hybrid retrieval →	formulation

Layer	Artifact classes	Consumption on device	Model in loop
R	Reasoning Units	grounding intent retrieval → instantiation	only instantiation only

§XVIII.3.8 Reference implementation pipeline (informative)

An illustrative end-to-end embodiment, without restricting alternatives:

1. **Publisher-side working store.** The WR Connect™ script maintains the semantic layer in a local SQLite database: pages with their normalized content, an interactive-element inventory per page (visible link/button texts with targets, headings — the grounding material for Semantic Anchors), graph nodes and edges with region classification, anchors, routes, Semantic Chunks, Reasoning Units, FAQ entries, and capability and WR Link™ declarations with their artifact hashes (the publisher's public surface, never secrets).
2. **Preparation passes.** Per page, a strong model produces the region classification, grounded anchors (validated against the element inventory, fail-closed), Semantic Chunks (self-contained, alias-annotated), and Reasoning Units for the intents this page serves. A site-level pass over the captured map produces cross-page routes and cross-page Reasoning Units. Preparation depth is an operator setting; the preparation may be expensive by design — it is performed once and amortized across every visitor interaction.
3. **Signed export.** The store is serialized deterministically into canonical declarative JSON, signed under the publisher key hierarchy, committed under the Catalog Head, and ingested with countersignature (§XVIII.3.1).
4. **Delivery.** Artifacts and updates reach the local orchestrator as BEAP/pBEAP capsules over the handshake-bound channel (§XVIII.3.6) — text-pure, sealed, code-free, vector-free (§XVIII.3.9).
5. **Local admission, merge, and indexing.** The orchestrator validates, admits under PoAC™, merges into the LSEM under §XVIII.4.2, and builds its local retrieval indexes per §XVIII.4.5 — including one-time local embedding of chunks and paraphrases with the device's own embedding model.
6. **Local PII enrichment.** Only now, and only locally, is the material enriched with the user's private context — names, addresses, order references, personal synonyms — as USER_PROVIDED/LOCAL_INFERRED assertions. PII never travels to the publisher or the WRC™; declared slots are resolved with private values at use time under identifier-only slot discipline.

§XVIII.3.9 Transport form of retrievable material: chunks, never vectors

Catalog artifacts and handshake-channel material MUST NOT contain embedding vectors, in either direction. The Knowledge Corpus and the retrieval surfaces of Reasoning Units travel as declarative text (chunks, aliases, paraphrases); every retrieval index is derived locally, on the visitor's device, at ingest (§XVIII.4.5).

This is an invariant, for four independent reasons:

1. **Model specificity.** Embedding vectors are meaningful only relative to one embedding model. Shipping vectors would either force a Canon-wide standardization of a single embedding model onto every visitor device — an authority this annex refuses to create — or deliver dead weight to every device using a different model.
2. **Determinism and signability.** The artifact is canonical, deterministic, and signed (§XVIII.3.1). Float vectors are none of these in any stable, cross-implementation sense, and they inflate the artifact by orders of magnitude.
3. **Boundary correctness.** The choice of embedding model is a property of the user's device and trust domain, not of the publisher's declaration. Local indexing at ingest is a one-time, seconds-scale cost even on constrained devices and keeps that choice where it belongs.
4. **Privacy class.** Embedding vectors admit reconstruction attacks; a vector-free channel removes the class entirely.

Corollary: locally derived indexes (vector or lexical) are local derived data. They are never synchronized to the publisher or the WRC™, and they fall under the LSEM protection discipline of §XVIII.4.4.

§XVIII.3.10 Site-level preparation passes

Per-page preparation (§XVIII.3.7) sees one page at a time. Four preparation concerns are site-level by nature and are defined here as passes over the complete captured map and the complete Layer-K/R output. Each pass is a preparation activity in the full sense of §XVIII.3.3: its output is declarative, provenance-labeled, catalog-committed material or a publisher-facing report; nothing here executes at visitor time.

1. Coverage gap analysis (publisher report, not catalog material). The preparation model enumerates the questions visitors of THIS application will plausibly ask and checks each against the prepared layers: answerable from a Reasoning Unit or chunk set, answerable only partially, or not answerable at all. The enumeration **MUST** include, beyond content questions, the **technical and account intents typical for the application class** — where to change a password, how to delete an account, how to update payment or address data, where to download invoices, how to reach a human, how to unsubscribe — instantiated for the specific site, not as a generic checklist. Unanswerable and partially answerable intents form the **Gap Report**: a publisher-facing to-do list naming each missing intent and, where inferable, the page where the answer would belong. The Gap Report is a quality instrument for the site itself; it is never shipped in the catalog artifact and never presented to visitors.

2. Conflict and duplication canonicalization. Where multiple pages yield chunks asserting the same fact, the pass merges them into one canonical chunk carrying all source refs (the duplicates are dropped, their aliases merged). Where pages yield **contradicting** assertions (two different return windows, two different prices for the same thing), the pass **MUST NOT** silently pick a winner: both chunks are retained, flagged, and surfaced to the publisher in a **Conflict Report** for an explicit editorial decision — the publisher's resolution becomes PUBLISHER_DECLARED, the losing assertion is removed by the publisher, never by the model. Rationale: contradictory grounding is the dominant cause of wrong retrieval-augmented answers; resolving it is an authorship act, and authorship belongs to the publisher (§XVIII.6.1).

3. Multilingual retrieval surfaces. Visitors ask in their language; the corpus is written in the site's language. The pass extends chunk aliases and Reasoning Unit paraphrases with translations into the languages the publisher declares (at minimum the site language plus English where they differ). Only the retrieval surfaces (aliases, paraphrases) are translated; chunk texts and resolutions remain single-source in the site language — the local model formulates the answer in the visitor's language at use time from the retrieved original. This keeps one canonical truth per fact while making retrieval language-robust.

4. Intent-to-capability mapping. Every Reasoning Unit whose resolution ends in an action the publisher offers as a declared capability (§XVIII.3.7 item 3) — contacting the publisher, registering, a declared WR Link™ automation — carries a reference to that capability in its `refs`. The consuming orchestrator MAY then present the governed capability as the offered next step of the answer, under the unchanged consent and PoAE™ chain; the mapping confers no authority (§XVIII.6.1) and is subject to label-independent consent (§XVIII.6.2).

Incrementality and honesty. Site-level passes are keyed to the state of their inputs (site fingerprint plus layer content) and re-run only on change. Reports (Gap, Conflict) are dated and reference the epoch state they were computed against; a stale report says so.

§XVIII.4 The Local Semantic Environment Model

§XVIII.4.1 Purpose and position

The LSEM is the orchestrator's persistent semantic model of the applications and environments the user interacts with: meaning, objects, relations, navigation paths, capabilities, workflow knowledge, Semantic Anchors, overlay declarations, the ingested Knowledge Corpus and Reasoning Units, locally added user information — and the local retrieval indexes derived from all of it.

Its advantage over per-request screen/DOM/LLM parsing is structural: parsing is repeated, probabilistic, and stale the moment it completes; the LSEM is validated once, authoritative at declaration level, incrementally updated by epoch, and queryable in milliseconds. Structural questions are answered without any model in the loop; content and intent questions are answered by local retrieval over indexed declarations plus a small formulation step — never by open-ended reasoning from raw interfaces.

Informative embodiment: a relational store with node/edge tables and recursive traversal, combined with a full-text index and a local vector index over chunks and paraphrases (e.g. SQLite with FTS5 and a vector extension), is sufficient. v1.2 note: v1.0 called vector search "optional and not a core requirement"; that posture is superseded — *retrieval-readiness* of the ingested material is now normative (§XVIII.4.5), while the concrete index technology remains an implementation choice, including lexical-only operation on constrained devices with the degradation posture declared.

§XVIII.4.2 Provenance classes and merge discipline

Every assertion in the LSEM carries exactly one provenance class:

- **PUBLISHER_DECLARED** — asserted by the publisher as declaration;
- **PUBLISHER_AI_INFERRED** — produced by publisher-side AI preparation and labeled as such;
- **LOCAL_INFERRED** — derived by local models or heuristics;
- **USER_PROVIDED** — supplied or corrected by the user;
- **PLATFORM_ATTESTED** — ingest countersignatures and platform scanning verdicts.

Local enrichment may extend but **MUST NOT** silently override publisher-declared assertions. Conflicts are represented as coexisting assertions with distinct provenance, resolved at use time under local policy, with the resolution disclosable to the user on request.

§XVIII.4.3 Lifecycle: epochs, revocation, purge

Provenance classes are the unit of lifecycle management. A catalog epoch advance invalidates and replaces **PUBLISHER_*** assertions selectively; local classes are preserved. Revocation of a WR Handshake™ triggers the declared retention/purge policy for that publisher's derived assertions and for local assertions derived solely from them. The handling of **LOCAL_INFERRED** assertions that build on superseded publisher epochs (retain as unconfirmed, re-validate, or discard) is a declared local policy choice; see Open Items.

§XVIII.4.4 Privacy by architecture

The publisher is authoritative for what its application means; it does not thereby learn what that meaning is worth in the user's private context. Private resolution — personal synonyms, preferences, files, mail, calendar, history, session context, private entity resolution, priorities, final parameter binding — occurs locally. Governing statement: **public intelligence is prepared globally; private meaning is resolved locally.**

Two consequences are normative:

1. **Query privacy.** Selective loading of semantic contexts is itself a signal. The Canon's default of no selection telemetry extends to PSL loading; load-class declarations and bundling strategies exist precisely so that fine-grained interest is not observable per request.
2. **LSEM protection.** The LSEM is structurally a record of the user's application world and is treated as high-sensitivity local data: encrypted at rest, compartmentalized per publisher, egress-governed by WR Guard™ like any other confidential store.

§XVIII.4.5 Ingest indexing and the grounded answer path

Indexing at admission (normative). After validation and PoAC™ admission, the orchestrator:

1. merges Layer S into the LSEM graph (§XVIII.4.2);
2. indexes Layer K (chunk texts and aliases) and Layer R retrieval surfaces (intents and paraphrases), plus the FAQ, into its local retrieval indexes: a lexical channel and a semantic channel (locally computed embeddings, §XVIII.3.9). On constrained devices one channel **MAY** degrade to the other; the degradation is a declared device posture, never silent;
3. on epoch advance, re-indexes only artifacts whose content-addressed ids changed — incremental by construction.

The grounded answer path (normative). A visitor request is served in this order:

1. **Deterministic short-circuit.** Pointer-at-element resolves through Semantic Anchors; an explicit navigation target resolves through declared routes. No model in the loop; the WR Pointer™ and WR Overlay™ consume the result directly under §XVIII.5.
2. **Hybrid retrieval.** Otherwise the request is matched against Reasoning Unit intents and Knowledge Corpus chunks over both channels; results are merged and bounded.
3. **Grounding Pack assembly.** The orchestrator assembles the bounded Grounding Pack: matched Reasoning Unit (if any), retrieved chunks, the relevant structural slice, the active Context Frame, and — locally only — the private context needed for instantiation.
4. **Local formulation.** The local model formulates or instantiates strictly within the Grounding Pack. Layer R units are instantiated (preconditions checked, steps bound to anchors/routes surfaced through §XVIII.5); Layer K material is formulated over.
5. **Attribution and honesty.** Answers are attributable on request to their provenance classes (§XVIII.5.3). When retrieval yields no sufficiently matching declared material, the assistant states that the publisher's declarations do not cover the question rather than improvising an answer in the publisher's name — the honest-boundary rule of §XVIII.6.5 applied to content.

Privacy (restated for this path). Queries, locally computed embeddings, retrieval indexes, and Grounding Packs never leave the user's trust domain; no per-query telemetry exists (§XVIII.4.4). Dispatch of the formulation step to another user device follows §XVIII.7 unchanged.

§XVIII.5 WR Overlay™ and Trusted Local Rendering

§XVIII.5.1 Declared, never delivered

Publishers never transmit user interface. Overlay behavior is declared: a declaration names a Semantic Anchor, a meaning, a target reference, and one overlay type drawn from a closed, locally defined overlay-type catalog. Rendering is performed exclusively by the orchestrator's trusted UI library. Publishers parameterize within the declared type; they never control presentation. This is the trust boundary between publisher intent / semantic declaration and local trusted rendering, and it deliberately inverts the model in which a publisher renders guidance into its own page. The visual material of overlays comes exclusively from the sources defined in §XVIII.5.4.

§XVIII.5.2 Invariants

1. **Non-occlusion.** A WR Overlay™ MUST NOT obscure, replace, or visually alter consequential original UI.
2. **Distinguishability.** Overlays are unmistakably marked as orchestrator-rendered, consistent with the Canon's marking obligations.
3. **Fail-closed anchoring.** If a Semantic Anchor does not resolve unambiguously against the live interface, no overlay is rendered at an assumed position; the unmapped-target fallback of Annex XI §XI.5 applies.

4. **Offer-never-action.** WR Overlay™ inherits Annex XI I2 unchanged: it may inform, indicate, and offer — never act. Consequential actions proceed only through capability, validation, consent, and PoAE™.

§XVIII.5.3 Context transparency

While publisher context is active, the user can see that it is active, from which verified publisher it originates, which semantic context (Context Frame) is in use, and why the orchestrator is making its current suggestions. Suggestions derived from publisher semantics are attributable to their provenance class on request.

§XVIII.5.4 Local overlay media library and positional guidance

WR Overlay™ rendering draws its visual material from exactly two sources:

1. **The local overlay media library** — the orchestrator's own trusted library of presentation primitives: arrows, path indicators, markers, highlights, step badges, and comparable directional and annotative elements. These primitives are local assets; publishers can request them by type (§XVIII.5.1), never supply them.
2. **Normalized publisher media, optionally:** media a publisher has uploaded to the Workflow Ready Cloud (WRC)™, normalized at ingest per the Canon's media pipeline, published hash-pinned, and referenced from overlay declarations by hash. The orchestrator renders such media only on hash match. Raw media never travels over the handshake channel.

Locally composed positional guidance. The PSL declares Semantic Anchors and positional references — where semantic targets live within the publisher's interface. The orchestrator resolves these declarations against the locally observed live interface and composes directional guidance from them entirely on the user's device: an arrow toward the invoice-download control, a highlighted path through a multi-step workflow, ordered step markers along a declared navigation route. The publisher declares where things are and what the target is; the guidance element itself — its geometry, placement, and rendering — is composed locally from the overlay media library. Every positioned element is subject to fail-closed anchoring (§XVIII.5.2): if a declared position or anchor does not resolve unambiguously, no guidance is rendered at an assumed location.

§XVIII.6 Semantic Authority Boundary and Security Model

§XVIII.6.1 Semantic authority is not consequence authority

Signatures on a PSL prove origin and integrity, never truth. A publisher is authoritative for the declared meaning of its own application; this authority confers no execution authority, no trust uplift, and no influence on consent. All consequential actions remain governed exclusively by the capability, validation, consent, and PoAE™ chain.

§XVIII.6.2 Attack class: semantic spoofing

A verified but malicious or compromised publisher may declare misleading semantics — for example, labeling a subscription trigger as a download link. Mitigations are structural, not heuristic:

1. **Label-independent consent.** The consent surface presented to the operator **MUST** derive from the validated action itself (intent hash, WYSIWYE per Annex IX), never from publisher-supplied labels or descriptions. The PSL may improve navigation and explanation; it never supplies the description of what the user authorizes.
2. **Platform scanning.** PSL artifacts are scanned by Fraud Watchdog™ in the WRC™ like all publisher material, under the suspension discipline of Annex XVII §XVII.3.3 (visible, auditable, reversible, never modification).
3. **Fail-closed anchoring** (§XVIII.5.2), which prevents misdirected overlays from being rendered on mismatch.

§XVIII.6.3 Staleness and anchor time-of-check/time-of-use

Declared semantics are epoch-versioned; the live interface is not. Divergence between the committed epoch and the rendered application is an expected condition, not an error path: anchor resolution is fail-closed, resolution failures are surfaced as declared conditions, and a resolution-failure signal is a legitimate trigger for the publisher to republish (§XVIII.3.4) — without per-user telemetry.

§XVIII.6.4 Overlay redressing

A local renderer over foreign UI can itself become a redressing instrument if declarations steer position or content too far. The invariants of §XVIII.5.2 — closed type catalog, non-occlusion, distinguishability — are the defense and are non-negotiable.

§XVIII.6.5 Honest boundary

The PSL eliminates interface ambiguity, not intention ambiguity. The local model remains responsible for interpreting what the user wants, and can be wrong. Nothing in this annex converts probabilistic intention interpretation into a guarantee; the guarantees end where declared semantics end, and consequence gating exists precisely because interpretation can fail.

§XVIII.7 The Personal Compute Dispatcher

§XVIII.7.1 Principle

The interaction endpoint does not have to be the execution endpoint. The Personal Compute Dispatcher places tasks among devices within the user's own or explicitly authorized compute environment: interaction device (typically the smartphone orchestrator, "Opty"), personal hosts, home servers, and — only under explicit External Service Admission per Annex X — designated cloud services. The dispatcher is a component of the personal orchestrator. It is not cloud routing: placement stays inside the user-controlled or user-authorized trust domain.

§XVIII.7.2 Placement modes

- **Automatic Dispatch** is permitted only for task classes the user has pre-authorized and only below a declared consequence floor.
- **Suggested Dispatch** applies otherwise: the orchestrator proposes offloading ("this task would run faster/more capably on your host"); the user confirms or continues locally.
- **Escalation** follows ratchet discipline: policy and user may tighten placement requirements, never loosen them below declared floors.

§XVIII.7.3 Boundaries between the user's own devices

Locality-independent trust (Annex X) applies between the user's own devices: data flowing to an execution endpoint is governed by the same classification, WR Guard™, and egress measures as any boundary crossing. Which data classes may reach which host is declared per host, not assumed from ownership.

§XVIII.7.4 Provenance, verification, return

Every dispatched task carries task provenance; every result carries result provenance — executing device identity, model identity, task hash — and is verified by the originating device before re-insertion into any publisher or workflow context. Dispatch is interruptible; results return to the originating device. The user can inspect where a task ran, why, with which data classes, under which model, and whether placement was automatic or confirmed.

§XVIII.7.5 Placement privacy

Placement is private. Publishers never learn where computation executed; the placement decision is invisible outside the user's trust domain.

§XVIII.8 Combined Operation: Brief Scenarios

Each scenario is stated briefly; the technical substance is in §§XVIII.3–XVIII.7.

Local-AI support. "Why was my order split into two shipments?" — answered locally through the grounded answer path (§XVIII.4.5): hybrid retrieval over the Knowledge Corpus and Reasoning Units, structural expansion through the Navigation Graph, formulation over the Grounding Pack plus private context; heavy analysis dispatched to the personal host if needed. The publisher runs no support AI and learns nothing.

Verified navigation. "OK Opty, show me where I download my invoice." — semantic target resolved from declared routes; WR Pointer™ navigates, WR Overlay™ marks the anchor; fail-closed on mismatch.

BEAP contact form. The declared BEAP contact form replaces web form and mail: text-pure, identity-classed BEAP capsules, screened by Fraud Watchdog™ at the publisher.

One-consent registration. Declared slots resolved from local PII under identifier-only discipline; one consent; confirmation handled as a governed action — fully deterministic through the semantic graph when both sides are in the WR system.

WR Link™ automation. A WR Link™ on the page triggers a declared, WCR-based automation — selected, never synthesized — under consent and PoAE™.

Cross-publisher local join (informative). "Compare my open orders across all shops" — a local graph join across provenance-tagged subgraphs plus private entity resolution; the result remains local; any outbound free text falls under the existing WR Guard™ egress discipline (Annex X).

§XVIII.9 Essential Combined Disclosure

The following combination is disclosed as of 10 August 2026:

(a) publisher-declared application semantics as cryptographically governed publisher material under a DNS-rooted catalog commitment chain with ingest countersignature and public auditability; (b) asymmetric publisher-side semantic preparation whose output crosses the trust boundary only as validated declarative artifacts, never as executable content; (c) a persistent Local Semantic Environment Model merging publisher-derived and locally derived meaning under explicit provenance classes, with selective epoch invalidation and revocation purge; (d) strict separation of semantic authority from consequence authority, with consent surfaces derived label-independently from validated actions; (e) WR Overlay™ rendering performed exclusively by the local trusted UI from declared intent, under non-occlusion, distinguishability, and fail-closed anchoring; (f) execution-location-aware orchestration across user-owned compute with consent-graded placement, dual task/result provenance, and placement privacy toward publishers; and (g) the resulting four-authority separation — the publisher knows the application; the user orchestrator knows the user; the dispatcher knows where computation should run; the trusted local UI controls what the user sees — governed end to end by one proof discipline.

Additionally disclosed as of 14 August 2026:

(h) the organization of publisher-side preparation output into three consumption-defined layers with a normative consumption binding — structural declarations consumed by deterministic lookup, a Knowledge Corpus of atomic self-contained Semantic Chunks consumed by local hybrid retrieval, and Reasoning Units carrying precomputed publisher-side reasoning consumed by intent retrieval plus local instantiation; (i) the transport invariant that retrievable material crosses the trust boundary exclusively as signed declarative text — chunks, aliases, and paraphrases, never embedding vectors — with every retrieval index derived locally on the visitor's device at ingest by the device's own embedding model; and (j) the grounded answer path: deterministic short-circuit through declared anchors and routes first, then local hybrid retrieval, bounded Grounding Pack assembly, local formulation strictly within the pack, and an honest-boundary refusal when declared material does not cover the question.

Individual elements are acknowledged as existing prior work in §XVIII.0.2, which covers this entire annex. The disclosed contribution is this combination and its governance.

§XVIII.10 Open Items

1. **Semantic Anchor resolution contract** — the technical form of anchors (structural hash, path, region geometry, semantic selector) and a graded multi-strategy resolution scheme with declared confidence.
2. **Epoch coupling of local enrichment** — normative default for LOCAL_INFERRED assertions built on superseded publisher epochs (retain-as-unconfirmed vs. re-validate vs. discard).
3. **Context Frame specification** — schema, lifetime, signature binding, and its relation to session sync across the user's devices.
4. **LSEM replication** — partitioning and synchronization of the LSEM across the user's own devices when the dispatcher separates interaction and execution endpoints; replication of locally derived indexes versus re-derivation.
5. **Semantic context vocabulary** — the open registry schema for context types (product page, checkout, registration, account, order, returns, support, workflow step) rather than a closed normative list.
6. **Dispatcher task-class taxonomy** — the declared task classes and their default consequence floors.
7. **Chunking granularity contract** — normative bounds and quality tests for Semantic Chunks (size envelope, self-containment test, alias budget) and for Reasoning Unit paraphrase budgets.
8. **Retrieval confidence floor** — the declared threshold semantics below which the honest-boundary rule of §XVIII.4.5(5) applies, and its user-visible presentation.
9. **Benchmark protocol** — the standard form of the accuracy-by-preparation falsification benchmark (§XVIII.3.3 informative): question-set construction, metrics (retrieval hit rate, groundedness, navigation-target success), and reporting.
10. **Gap-intent taxonomy** — the open registry of application-class intent families (account, billing, privacy, support, fulfillment) the coverage gap analysis instantiates per site.
11. **Conflict resolution UX contract** — the minimal editorial actions (keep A, keep B, write C) and how the resolution propagates into Layer K.
12. **Declared language set** — where the publisher declares its retrieval languages and how the set versions with epochs.

§XVIII.11 Extension Points

- Additional provenance classes beyond §XVIII.4.2, added only with a declared merge rule.
- Additional overlay types in the closed local catalog; publisher-side proposals for new types are admitted as Canon amendments, never at runtime.
- Additional Semantic Chunk kinds and Reasoning Unit fields, admitted as Canon amendments with declared indexing semantics, never at runtime.
- Extension of Context Frames to multi-party sessions (interaction with the group-session work).
- Dispatcher extension to attested non-personal compute under External Service Admission profiles.

- Accessibility profiles binding PSL structures to assistive-technology conventions.
-

§XVIII.12 — Opty Adaptive Pointer Menu and Persistent Pinboard

1. System-Wide Adaptive Pointer Menu

Optirando may extend the system-wide pointer or mouse interaction with an Adaptive Menu. A visually distinct, color-highlighted Adaptive area is integrated into the pointer menu and presents actions derived from the user's current context.

The available actions may depend on the current application, page, object, publisher context, handshake state, workflow state, or other information available to the Orchestrator.

The Adaptive area remains visually identifiable while its individual actions may change dynamically. Selected adaptive actions can be pinned by the user and thereby promoted from temporary contextual suggestions into persistent personal shortcuts.

2. Opty as Persistent Personal Action Surface

Opty is a small floating owl companion that provides direct access to the user's pinned actions.

Opty is designed to remain visually small, freely movable, and dockable to screen edges. Opty continuously carries a pinboard in its visible hands/wings. The pinboard is a separate control surface and contains only pinned actions.

The pinboard adapts horizontally to the number of pinned items. It may become wider than Opty's body while Opty continues to visibly hold the board. For larger collections, compact overflow mechanisms may be used instead of continuously increasing its size.

Pinned items may represent, among other things:

- direct functions or commands;
- #tags;
- WCR recordings;
- configured automations or workflows;
- actions previously discovered through the Adaptive Pointer Menu.

3. Persistent Pins During Sleep

Opty's sleep state is a character and assistant state, not a deactivation of the pinboard.

Pinned icons remain visible and directly usable while Opty is sleeping. This allows frequently used actions to remain available without first opening another menu or waking the assistant.

A pinned action may therefore be executed directly by clicking its icon even while Opty is visually asleep.

4. Voice and Click Activation

Opty supports both direct interaction and voice-triggered execution.

A wake phrase such as:

"OK Opty"

may activate the assistant for subsequent interaction.

A combined wake-and-command expression may directly address a pinned semantic action, for example:

"OK Opty, run #Work."

The same pinned action may alternatively be executed by clicking its corresponding icon on Opty's pinboard.

Thus, voice and pointer interaction address the same persistent action surface rather than representing separate function sets.

5. Interaction Principle

The resulting interface deliberately separates two complementary concepts:

Adaptive Pointer Menu → **contextual and temporary**. Provides actions relevant to what the user is currently doing.

Opty Pinboard → **personal and persistent**. Provides immediate access to actions the user has deliberately pinned.

An action may therefore follow the interaction path:

Discover in Adaptive Menu → Pin → Persist on Opty → Execute by click or voice.

This separation allows contextual intelligence to remain available directly at the pointer while keeping Opty compact, personal, persistent, and continuously accessible.

§XVIII.13 Version History

- **v1.0 — 10 August 2026**. Initial version, published under the designation WR Connect™ (the publisher-side wr-connect script is its reference embodiment). Defines the Publisher Semantic Layer as governed publisher material generalizing the Annex XIV Scope Graph; Asymmetric Preparation with explicit accuracy-by-preparation design goal (low-power local AI operates accurately on publisher-side strong-reasoning output), method-agnostic authoring (manual, local AI, cloud AI), and review-based provenance labeling; explicit preparation output inventory (Navigation Graph, Q&A scenario corpus, FAQ, Semantic Anchors/position markers, semantic navigation aids, BEAP contact form capability, WR Link™ declarations bound to WCR recordings) with informative SQLite → BEAP → LSEM reference pipeline and local PII enrichment (§§XVIII.3.7–3.8); the Local Semantic Environment Model with provenance classes and lifecycle; introduces WR Overlay™ as the branded designation for the Augmented Overlay ("Augmented Overlay" remains the

descriptive technical term; both refer to the same construct); Semantic Authority Boundary with the semantic-spoofing attack class and label-independent consent; the Personal Compute Dispatcher with placement modes, dual provenance, and placement privacy; local overlay media library with hash-pinned normalized publisher media and locally composed positional guidance (§XVIII.5.4); brief scenarios (local-AI support, verified navigation, BEAP contact form, one-consent registration, WR Link™ automation); consolidated best-knowledge prior-art acknowledgment in the Claims Discipline (§XVIII.0.2, no element-by-element differentiation); essential combined disclosure.

- **v1.1 — 12 August 2026.** Refines the Opty™ interaction model by separating the system-wide Adaptive Pointer Menu from Opty's personal persistent pinboard; introduces a visually distinct, color-highlighted Adaptive area within the pointer menu for context-derived actions; allows selected Adaptive actions to be pinned to Opty; defines Opty as a tiny floating, freely movable, and dockable owl companion that continuously carries the user's pinned actions; defines adaptive horizontal pinboard growth, including extension beyond Opty's body while remaining visibly carried by Opty; preserves pinned actions as visible and directly usable while Opty is sleeping; adds direct click execution of pinned actions; and adds voice activation and execution through commands including "OK Opty" and combined wake-and-action expressions such as "OK Opty, run #tag."
- **v1.2 — 14 August 2026.** Restructures the preparation output inventory (§XVIII.3.7) into three consumption-defined layers and makes the consumption binding normative — prepared material a local mechanism cannot look up or retrieve transfers no value. **Adds the Knowledge Corpus (Layer K, Semantic Chunks):** the application content itself in retrieval-ready, self-contained, alias-annotated form — previously absent from the inventory, which left visitor-side content questions without any grounding material. **Supersedes the Q&A scenario corpus by Reasoning Units (Layer R)** — reasoned invariant change: question-answer surface forms without retrieval surfaces and reference discipline transferred no usable value to small local models; Reasoning Units carry the precomputed reasoning (intent, paraphrases-for-retrieval, preconditions, structure-bound steps, grounded resolution, refs) that the local model only instantiates. **New transport invariant "chunks, never vectors" (§XVIII.3.9)** with its four reasons (model specificity, canonical signability, boundary correctness of local embedding, embedding-inversion privacy class). **New ingest indexing and grounded answer path (§XVIII.4.5):** local hybrid indexing at admission, deterministic short-circuit first, bounded Grounding Packs, honest-boundary rule for uncovered questions, incremental re-indexing by content-addressed ids. **LSEM posture updated (§XVIII.4.1):** retrieval-readiness is normative; index technology remains an implementation choice (supersedes the v1.0 "vector search is optional" wording, reasoned). **Review economics (§XVIII.3.3):** bulk adoption acts and edit-as-adoption are legitimate; per-item ceremony must not be the only tooling path. Grounding rules for anchors and routes (fail-closed against uncaptured elements/pages) lifted from embodiment practice into the inventory (§XVIII.3.7). New definitions: Semantic Chunk, Reasoning Unit, Grounding Pack, Hybrid Retrieval (§XVIII.2). Reference pipeline updated with element inventory, preparation depth, and local indexing (§XVIII.3.8). Local-AI support scenario aligned to the grounded answer path (§XVIII.8). Second essential-combined-disclosure paragraph dated 14 August 2026 covering (h) the three-layer consumption binding, (i) the vector-free transport invariant with local ingest-time embedding, and (j) the grounded answer path (§XVIII.9).

Open Items extended (chunking granularity contract, retrieval confidence floor, benchmark protocol). Editorial: v1.1 numbering collision resolved — the Opty Adaptive Pointer Menu section becomes §XVIII.12, a single Version History becomes §XVIII.13, and the Amendment Discipline reference (§XVIII.0.3) now points to §XVIII.13; the duplicated v1.0 history entry is removed. No content of the Opty section changes.

- **v1.3 — 14 August 2026.** Adds §XVIII.3.10, the site-level preparation passes: coverage gap analysis producing a publisher-facing Gap Report whose intent enumeration normatively includes site-specific technical and account intents (password change, account deletion, payment/address updates, invoice access, human contact, unsubscribe); conflict and duplication canonicalization with the rule that contradictions are surfaced for publisher decision, never silently resolved by the model; multilingual retrieval surfaces (aliases and paraphrases translated, canonical texts single-source); and intent-to-capability mapping under the unchanged consent chain. Open Items extended (gap-intent taxonomy, conflict resolution UX contract, declared language set).

1

WR Connect™

Governed Publisher Semantics and Personal Compute Orchestration

Optirando™ Canon · Annex XVIII · Defensive Publication / Prior-Art Disclosure · 10 August 2026

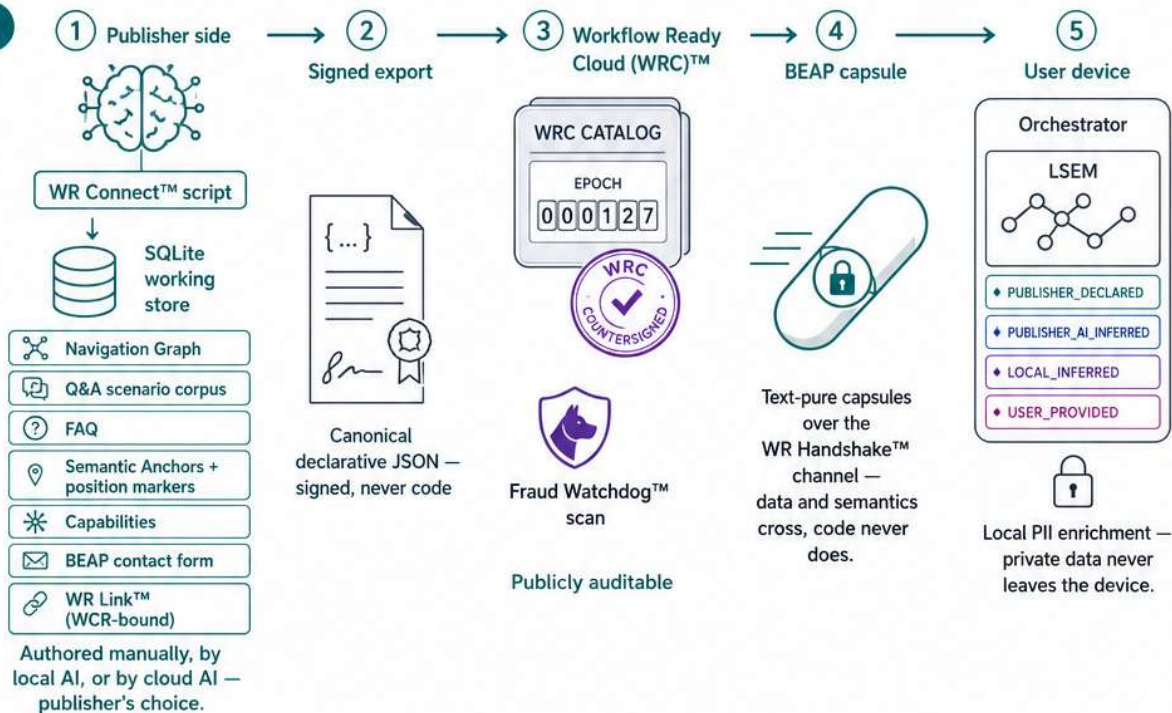


Publisher-provided semantics instead of screen guessing.



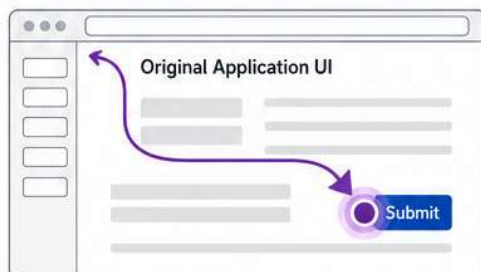
Strong intelligence prepares the environment — so low-power local AI operates accurately.

2



3

WR Overlay™



→ Rendered only by the local trusted UI from the local overlay media library (arrows, paths, markers)

⦿ Publisher declares intent + Semantic Anchors — never delivers UI

🛡️ Fail-closed anchoring · Non-occlusion

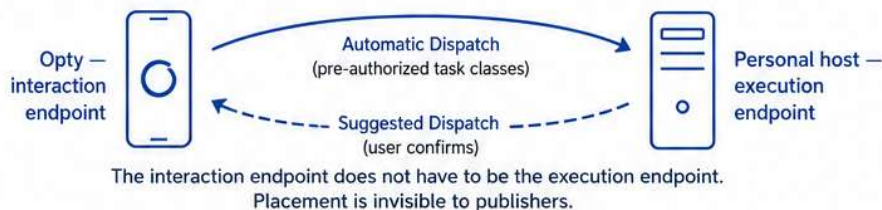
🖱️ WR Pointer™

4



Four authorities that never collapse — governed end to end by WR Handshake™, BEAP™, PoAC™, PoAE™.

5



6



Optirando™ · WR Connect™ · WR Overlay™ · WR Pointer™ · WR Link™ · WR Handshake™ · BEAP™ · PoAC™ · PoAE™ · Fraud Watchdog™ · Workflow Ready Cloud (WRC)™ are designations of the Optirando™ Canon.

Opty UI Concept

System-wide Adaptive Pointer Menu + Personal Pinned Actions

The pointer menu adapts to what you're doing right now.
Opty keeps your favorite actions always within reach.



- Tiny & Floating
- Movable
- Dockable
- Always in Reach

1 Wake Opty

1 Sleeping (Pinned icons stay visible)



Opty is sleeping, but your pinned icons remain visible and usable.

2 Activate by Voice (Two ways)

Say to wake Opty

OK Opty!



Voice: Say "OK Opty" to wake.

Say to run an action

OK Opty, run #Work



Voice: Say "OK Opty, run #tag" to execute a pinned action.

3 Activate by Click / Tap



Click a pinned icon directly to trigger the action.

4 Opty is Ready



Opty is awake and ready.
Pinned actions are at your fingertips.

★ Pinned icons stay visible, even while Opty sleeps, so you can always wake Opty or run actions by voice or click.

2 Adaptive Pointer Menu



- Lives in the system-wide pointer menu
- Context-sensitive actions near the cursor
- Color-highlighted Adaptive area
- Based on current target, app, page, handshake, and workflow context
- Some actions can be pinned

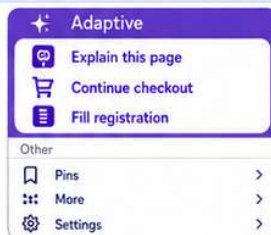
VS

Opty (Your Pinboard)



- Tiny floating owl companion
- Freely movable and dockable
- Shows only your pinned actions
- Pins can trigger features, #tags, WCR recordings, and automations
- Shield/pinboard grows horizontally as more pins are added

3 Pointer Menu (Example)



Adaptive area = context-derived suggestions

Same position and accent color — only the actions change based on context.

4 Opty Pinboard Adapts to Your Needs



Docked to edge



5 From Discover to Persistent Shortcut



1 Discover

A useful action appears in the Adaptive area based on current context.



2 Pin

The user pins that action directly from the Adaptive area.



3 Use through Opty

The action becomes a persistent shortcut on your Opty pinboard.



Scope (where it can appear)



Pinned actions can keep their intended scope.

6 Design Principles

- ✓ Adaptive = contextual and temporary
- ✓ Opty = personal and persistent
- ✓ Stable layout reduces misclicks
- ✓ Accent color means context-derived
- ✓ Pins should stay compact and glanceable

At a Glance



Voice Activation
Say "OK Opty" to wake Opty.



Click Activation
One click or tap on Opty wakes it.



System-wide Pointer Menu
Context-aware near the cursor.



Personal Pinboard
Your shortcuts, always within reach.



Secure & Private
Your pins stay with you across apps.

Separate contextual actions from personal shortcuts.

The pointer menu adapts to what you're doing now.
Opty keeps your favorite actions always within reach.



Smarter workflows.
Faster actions.
Better decisions.



Built for your mission.
Configured for your success.



Always ready.
Whenever you need it.



Optirando